CHAPTER SEVEN
SDLC vs UML

Software Development Life Cycle

- The UML is largely process-independent, meaning that it is not tied to any particular software development life cycle.
  However, to get the most benefit from the UML, you should consider a process that is
  - Use case driven
  - Architecture-centric
  - Iterative and incremental

Use case driven

- It means that use cases are used as a primary artifact for establishing the desired behavior of the system, for verifying and validating the system's architecture, for testing, and for communicating among the stakeholders of the project.

Architecture-centric

- It means that a system's architecture is used as a primary artifact for Conceptualizing, constructing, managing, and evolving the system under development.

An iterative process

- It is one that involves managing a stream of executable releases.

An incremental process

- It is one that involves the continuous integration of the system's architecture to produce these releases, with each new release embodying incremental improvements over the other.

- Together, an iterative and incremental process is *risk-driven*, meaning that each new release is focused on attacking and reducing the most significant risks to the success of the project. This use case driven, architecture-centric, and iterative/incremental process can be broken into phases.

- **A *phase*** is the span of time between two major milestones of the process, when a well-defined set of objectives are met, artifacts are completed, and decisions are made whether to move into the next phase.

- As below figure shows, there are **four** phases in the software development life cycle: inception, elaboration, construction, and transition. In the diagram, workflows are plotted against these phases, showing their varying degrees of focus over time.

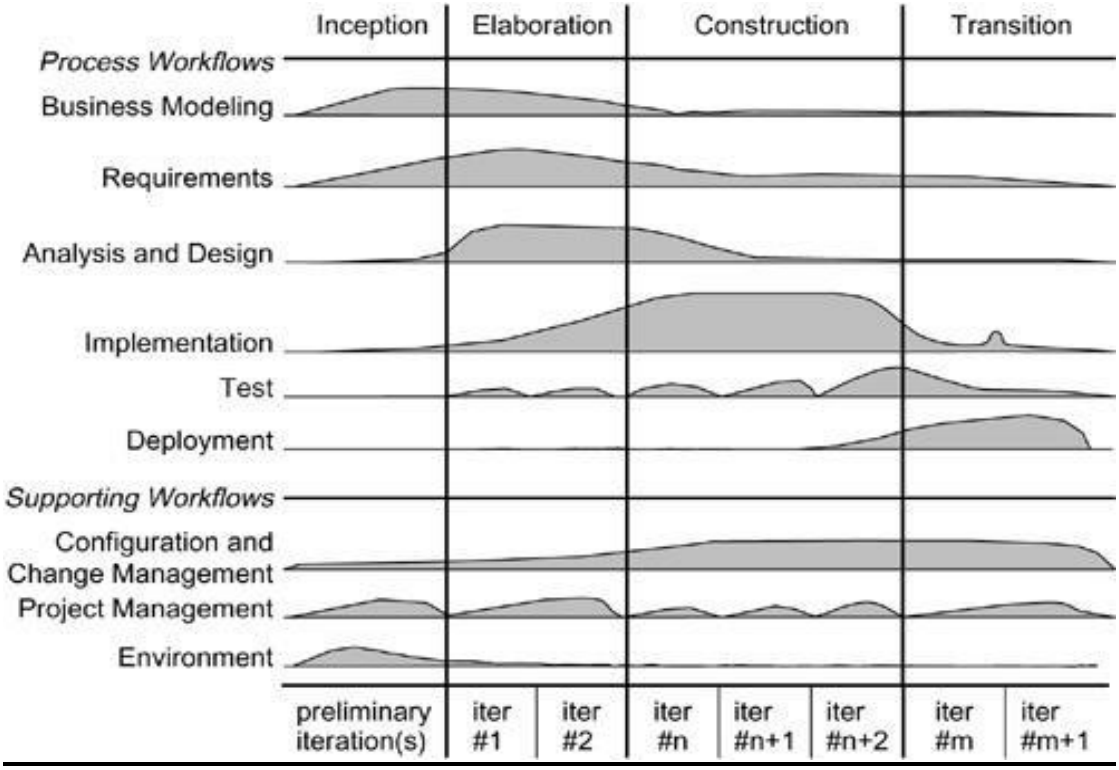


Fig: Figure Software Development Life Cycle

Inception phase:
It is the first phase of the process
When the seed idea for the development is brought up to the point of being• at least internally• sufficiently well-founded to warrant entering into the elaboration phase.

Elaboration phase:
It is the second phase of the process
When the product vision and its architecture are defined.
In this phase, the system's requirements are articulated, prioritized, and baselined.
A system's requirements may range from general vision statements to precise evaluation criteria, each specifying particular functional or nonfunctional behavior and each providing a basis for testing.

Construction phase:
It **i**s the third phase of the process.
When the software is brought from an executable architectural baseline to being ready to be transitioned to the user community.
The system's requirements and especially its evaluation criteria are constantly reexamined against the business needs of the project, and resources are allocated as appropriate to actively attack risks to the project.

Transition phase:
It is the fourth phase of the process.
When the software is turned into the hands of the user community.

The software development process end here, for even during this phase, the system is continuously improved, bugs are eradicated, and features that didn't make an earlier release are added.

One element that distinguishes this process and that cuts across all four phases is an iteration.
An iteration is a distinct set of activities, with a baselined plan and evaluation criteria that result in a release, either internal or external.
This means that the software development life cycle can be characterized as involving a continuous stream of executable releases of the system's architecture.
It is this emphasis on architecture as an important artifact that drives the UML to focus on modeling the different views of a system's architecture.